Lab 2

The goals of this lab are to practice installing a new shoe.

if you did not complete the prior lab. Go back and complete it before continuing here.

**Background**
Charlie is both an app and an API. A Charlie plugin is a concrete class that implements a Java interface based on the API. The plugin gets installed by Charlie dynamically at runtime using Java reflection. The plugin might be specified via the properties file, charlie.props. Some properties that are not in charlie.props can also be specified by virtual machine (VM) options in the runtime configuration of IntelliJ or through the java command line. If a Charlie property is not defined in charlie.props or VM options, Charlie uses default behaviors.

`IShoe` is the most basic plugin. A game cannot be played without an `IShoe` as it defines behaviors which the dealer uses to get cards. The `IShoe` interface is modelled on a casino shoe which contains the cards. An `IShoe` shuffles the cards and furthermore, indicates when a shuffle is pending, i.e., it is running out of cards, which in the GUI causes the burn card to appear. There are various `IShoe` implementations in Charlie, most of them for testing purposes.

In this lab you will just install a new `IShoe` plugin that extends another shoe and playtest it.

**Tasks**
Part I - Review the base shoe.

Study `charlie.plugin.IShoe` and `charlie.shoe.Shoe01`, which implements `IShoe`.

Part I - Install Charlie into the local Maven repository.
After initially downloading and opening Charlie in IntelliJ, Charlie is just a standalone project. Its API needs to be installed so that other projects can use it.

1. Open IntelliJ then open Charlie.
2. In the rightmost panel on IntelliJ, under , do Maven > Charlie > Lifecycle > Install > Modify Run Configuration…
3. Under Java Options > Modify > SkipTests. The Maven options panel should look like the following:

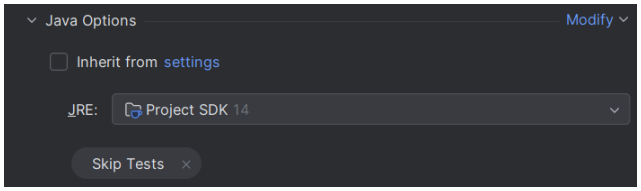

Figure 1.

4. Disclose “Run Configurations” under m Charlie then right-click m Charlie [install] > Run. This causes Charlie to rebuild itself and deploy the API into the local Maven repository on the local host.

Part II - Create a new plugin project.
This project will contain all your subsequent codes, plugins and test cases.

1. In IntelliJ, do File > New Project… and select Language: Java and Build system: Maven. Name your project <*your-name*>CharliePlugins (henceforth called the “plugins project”). For instance, my project is called RonzCharliePlugins.

2. Create the plugins project in a new window, i.e., keep the Charlie project open.

3. In the plugins project, open the project object module or `pom.xml` file which specifies the build dependencies, among other things. The plugins project is dependent on Charlie.

   Add to dependencies section--or create one, if it does not already exist--the Charlie dependency. It should look something like this:

```
17    <dependencies>
18        <dependency>
19            <groupId>charlie</groupId>
20            <artifactId>charlie</artifactId>
21            <version>3.1-SNAPSHOT</version>
22        </dependency>
23    </dependencies>
```

   Figure 2.

   This points the plugins project to Charlie.

4. Inside the plugins project, inside project scope, that is, inside the <project> and </project> tags, copy-paste to the clipboard for later (see below) this declaration the declaration (remember, yours won’t be RonzCharliePlugins but <*your-name*>CharliePlugins):

```
7    <groupId>RonzCharliePlugins</groupId>
8    <artifactId>RonzCharliePlugins</artifactId>
9    <version>1.0-SNAPSHOT</version>
```

   Figure 3.

   You will need this later.

Part III - Make a new shoe.

1. In the plugins project, create a new package, <*your-name*>.plugin. Eg, mine is called ronz.plugin.

2. Create a new Java class called, `MyShoe`, in the above package and make it extend `charlie.shoe.Shoe01`.

Part IV - Install the shoe plugin.

1. In the rightmost panel on IntelliJ of the plugin project, under , do Maven > *plugin-project-name* > Lifecycle > double-click Install.

2. Open `pom.xml` of Charlie and in the dependencies section, add the plugins project as a new dependency, ie, between `<dependency>...</dependency>`, input the text you copy-pasted to the clipboard in Figure 3.

3. Then, under , do > Reload All Maven Projects.

   This points Charlie to the plugins project.

Part V - Playtest the shoe plugin.

1. In Charlie do Run > Edit Configurations…

2. In the dialog that pops up, click (Add New Configuration) > Application and…
   a. In the Name field input `MyServer`.

   b. In the Main class field input `charlie.server.GameServer`.

   c. Click Modify options drop-down and select Add VM options.

   d. In the VM options field input the fully qualified classname of your shoe, i.e., -Dcharlie.shoe=<*your-name*>.plugin.MyShoe

      This last step directs Charlie to use your plugin shoe, not the default plugin shoe.

3. Click the run arrow for MyServer.

4. Right-click on charlie.GameClient > Run ‘GameClient.main()’.

   After logging in but before playing, check the server log. If the shoe was installed properly, expect to find something like:

   `shoe property 'charlie.shoe' = '`*`<your-name>`*`.plugin.MyShoe'`

```
shuffling burn index = 35
shoe: <your-name>.plugin.MyShoe: C5 DK S10 HK D9 DQ D3 D5 C4 C9
D8 CA C3 HQ H6
```

The cards, C5, DK, S10, etc., will be different for you because the shoe is randomly seeded each time the server runs.

> You have a bug and cannot proceed if you find a class not found exception, class cast exception, etc. in the diagnostic log. Most likely it's a typo, the local Maven repository has not been loaded as indicated above, or some other problem.

5. Playtest Charlie to the first burn card and take a screenshot.

**Deliverables**

1. Upload the client log file.
2. Upload the server log file.
3. Upload MyShoe.java.
4. Upload the screenshot.